



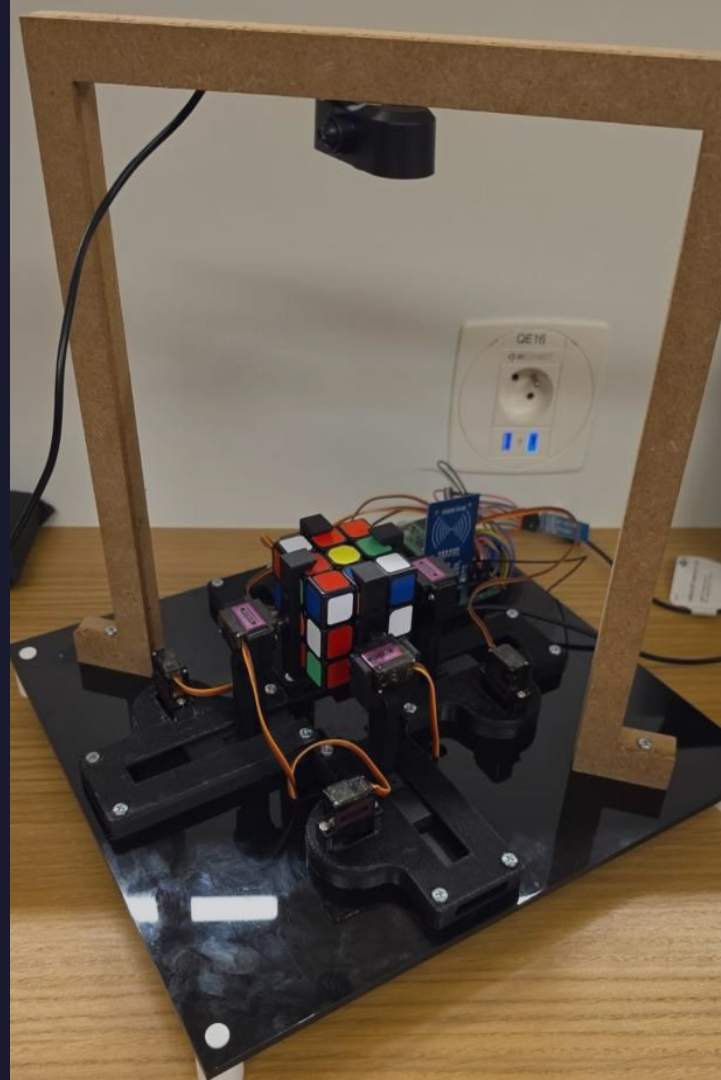
A2073 — ROBOTIQUE (PART 2) · EPHEC 2025-2026

Rubik's Cube Solver Robot

Conception et réalisation d'un robot autonome
de résolution du Rubik's Cube 3×3

EL YAZAMI Yassine MAZID Mohamed ELHAMAYDA Amer

Groupe B02 — Classe 2AU — Juin 2026





Un défi mécanique, électronique et logiciel

43×10^{18}

configurations possibles

21

mouvements max.

~ 20 s

temps de résolution de
notre robot

Objectif du projet

- Concevoir un robot qui scanne, calcule et exécute la solution d'un Rubik's Cube sans intervention humaine
- Combiner trois domaines : mécanique (4 bras), électronique embarquée (PCB sur mesure) et logiciel (PC + Pico)
- Identification par badge RFID, supervision par interface PC et communication Bluetooth



Les composants du système

- **Raspberry Pi Pico W** — Microcontrôleur — firmware MicroPython
- **HC-05 Bluetooth** — Liaison PC ↔ Pico (UART, 38 400 bauds)
- **MFRC522 RFID** — Identification par badge (SPI)
- **OLED SSD1306** — Affichage état & progression (I2C)
- **8× servos MG90S** — 4 rotations + 4 pinces (PWM 50 Hz)
- **LED RGB + bouton** — État visuel & commande manuelle

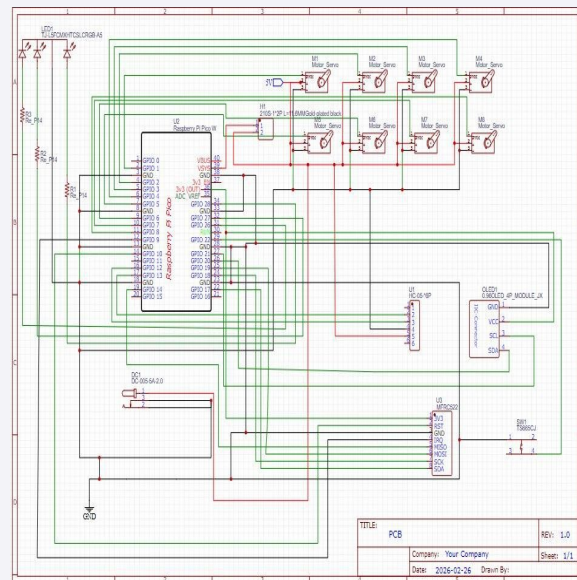
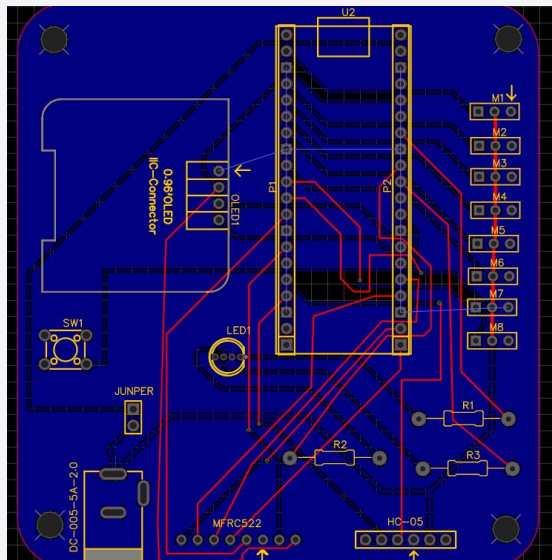


Schéma électrique complet

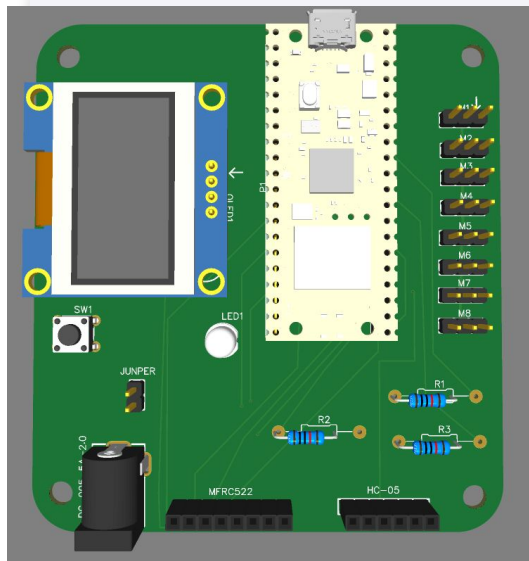
Alimentation 5 V séparée pour les servos : évite les chutes de tension lors des mouvements simultanés.



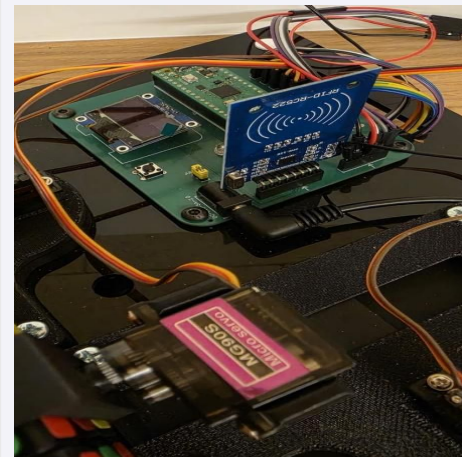
Un PCB conçu sur mesure



Routage



Rendu 3D

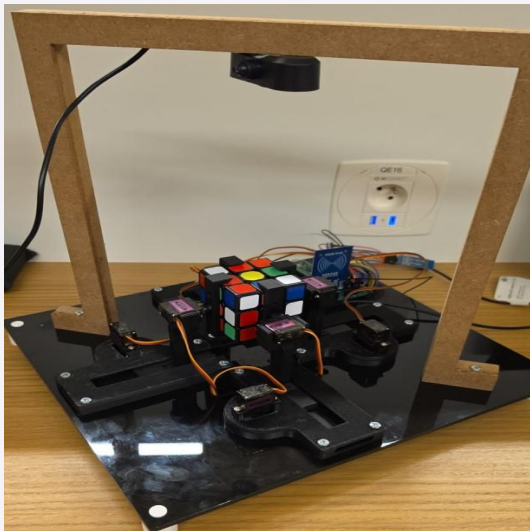


PCB soudé & assemblé

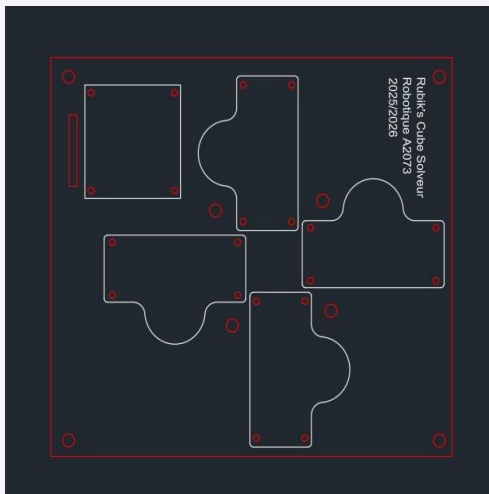
81,15 × 88,14 mm · 21 composants · 2 couches · 141 pastilles · fabriqué via JLCPCB



Une structure en croix à 4 bras



Robot assemblé



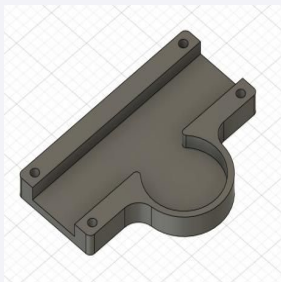
Plan de découpe laser

Principe

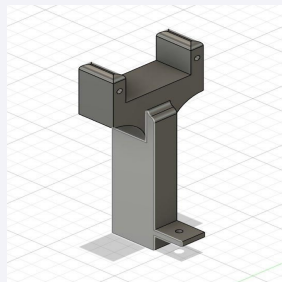
- 4 bras en croix face aux faces F, B, L, R du cube
- Chaque bras = 1 servo de rotation + 1 servo de pince
- Base découpée au laser : positionnement précis et reproductible
- Pièces des bras imprimées en PLA (Fusion 360)
- Caméra USB fixée au-dessus du centre



Pièces modélisées sous Fusion 360



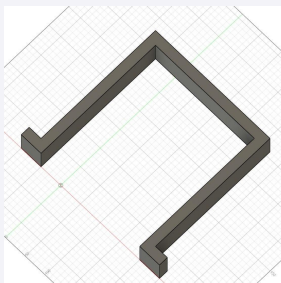
Platine support servo



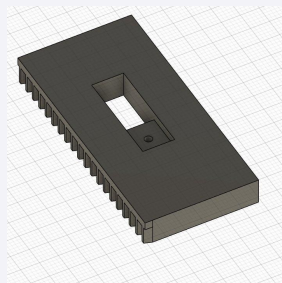
Colonne verticale



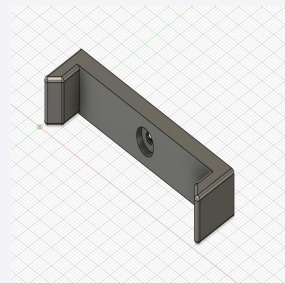
Engrenage (trou en D)



Cadre structural en U



Support de caméra



Pince de saisie

Servomoteurs MG90S & calibrage par bras



Paramètre	Bras L	Bras R / F	Bras B
ROT_CENTER (neutre)	86°	90°	94°
GRIP_HOLD (saisie)	20°	18° / 30°	35°
GRIP_OPEN (relâche)	100°	100°	100°
Temps / mouvement	0,12 s	0,12 s	0,12 s

Pourquoi un calibrage individuel ?

- Chaque bras a ses propres constantes :
compense les asymétries mécaniques de
l'impression 3D
- Couple 1,8 kg/cm à 5 V — engrenages métal
- U et D (pas de bras dédié) : basculement du
cube + rotation de la face accessible

Chaque mouvement (R, U', F2...) → **rotation CW/CCW** → **relâche pince** → **retour centre** → **saisie**



Firmware MicroPython : 8 modules

main.py Boucle principale : RFID, bouton, Bluetooth

servo_controller.py 4 bras / 8 servos, calibrage individuel

state.py Machine à états : IDLE / MOVING / SCAN

bluetooth.py UART 0, buffer 512 octets, parsing du protocole

rfid + oled + led Badge, affichage 4 lignes, LED RGB d'état

Le bouton physique

- Appui court (< 0,8 s) : ouvrir / serrer les pinces
- Appui long (> 0,8 s) : scan automatique des 6 faces

Sécurité RFID

- Liste blanche de badges autorisés
- Badge refusé → LED rouge, commandes bloquées, ERR:NOT_AUTHORIZED envoyé au PC



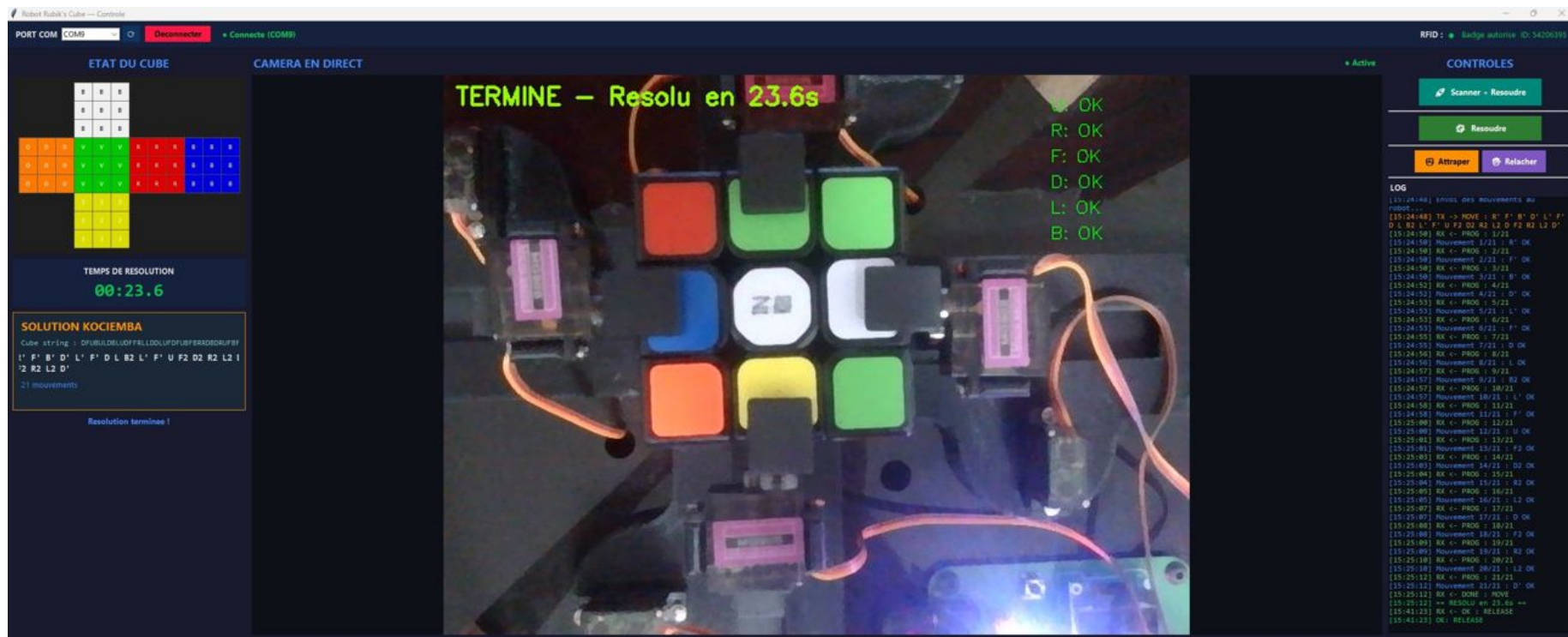
Protocole Bluetooth maison

START : TAG : PAYLOAD : END\n

Direction	Tag	Rôle
PC → Pico	MOVE / GRAB / RELEASE	Exécuter une séquence, serrer / relâcher les pinces
PC → Pico	START_SCAN / SCAN_ACK	Déclencher le scan, acquitter chaque face capturée
Pico → PC	SCAN_READY / SCAN_DONE	Face en position devant la caméra, scan terminé
Pico → PC	PROG:i/total / DONE	Progression mouvement par mouvement
Pico → PC	AUTH:id,0 1	Résultat de la lecture RFID

Buffer logiciel de 512 octets côté Pico : absorbe les longues solutions et filtre le bruit d'initialisation du HC-05.

L'interface de supervision Tkinter





Détection des couleurs (OpenCV / HSV)

1

Patch 17×17 px par case du cube

2

Couleur médiane comparée aux
plages HSV

3

9 captures successives par face

4

Vote majoritaire case par case

	Couleur	Teinte H	Conditions
	Blanc	0 – 180	$S < 60$ et $V > 170$
	Orange	6 – 22	testé AVANT le rouge
	Jaune	23 – 38	$S > 80$ et $V > 80$
	Vert	39 – 85	$S > 70$ et $V > 50$
	Bleu	95 – 135	$S > 70$ et $V > 50$
	Rouge	0-5 / 170-179	double plage circulaire



L'algorithme Kociemba



- Algorithme « two-phase » de Herbert Kociemba — tables précalculées embarquées localement (dossier twophase/), aucun service externe
- La solution est envoyée au Pico via Bluetooth (MOVE:...) puis exécutée mouvement par mouvement
- Après chaque étape : PROG:i/total → barre de progression + simulation en direct sur le CubeView

Résolution physique complète : **≈ 20 secondes**



Difficultés rencontrées & solutions

● Précision des servos

Calibrage individuel par bras (ROT_CENTER, GRIP_HOLD)

● Confusion orange / rouge

Orange testé avant le rouge + fallback par distance circulaire sur H

● Détection couleur instable

Vote majoritaire sur 7 captures au lieu d'un seuil fixe

● Bruit du HC-05 (« OK\r\n »)

Recherche du pattern START: dans le flux, buffer 512 octets



Conclusion

Objectif atteint — un robot 100 % autonome : scan → calcul →
exécution en ≈ 20 s

Place à la démonstration !

